



From Data to Solutions · Weekend 7

Distributed gradient descent: splitting the batch

Part 1 of 2: when the model fits and the data does not

Carlos Cotrini

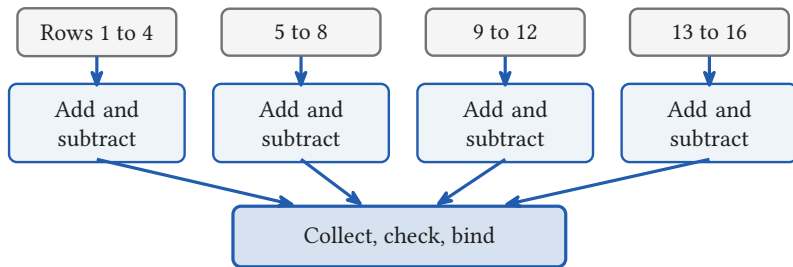


Section 1

The gradient of a mean



Paris, 1793: one task, twenty computers



- The same arithmetic in every hand, a different slice of the table
- Not a pin factory: nobody specialises in a step
- [Roegel 2010](#), from the payrolls: about twenty computers, not the sixty to eighty Prony recalled.

The sum splits, exactly

$$\nabla L(w) = \frac{1}{P} \underbrace{\sum_{p=1}^P g_p}$$

the only thing the workers must share

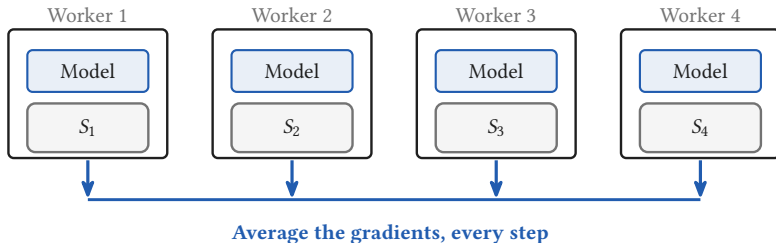
The batch, n examples



P workers, n/P examples each

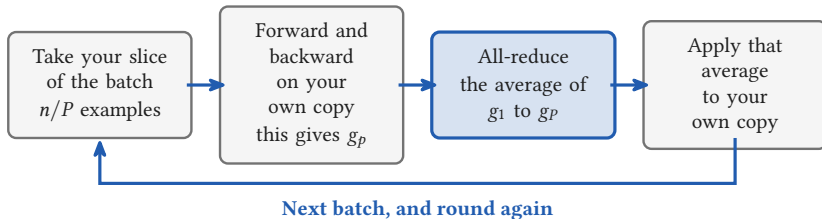
- Equal shards, so every worker holds the same number of examples
- You already have this from Weekend 0. Nothing here is new mathematics.

Every worker holds the whole model



- Data parallelism: replicate the model, split the batch
- The copies stay identical because every worker applies the same average
- This hour assumes the model fits on one device

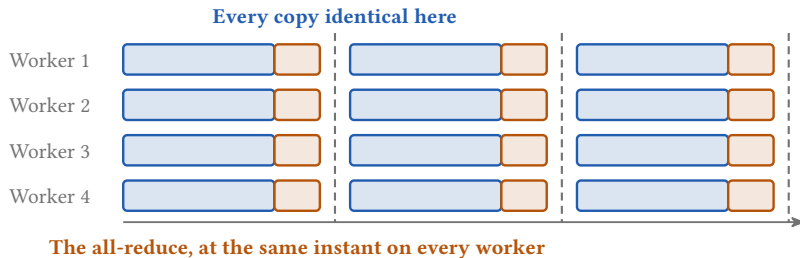
One step of training, run on every worker at once



Three of the four stages are local. The all-reduce is the one that touches the network.

- Every worker runs this same loop, at the same time, on its own slice
- Split the data, aggregate the gradient, and repeat

One all-reduce per step, for every step of the run



- The average lands before the weights move, so every step starts from one model
- Local SGD relaxes this and aggregates every few steps. Everything in this hour is the synchronous version.

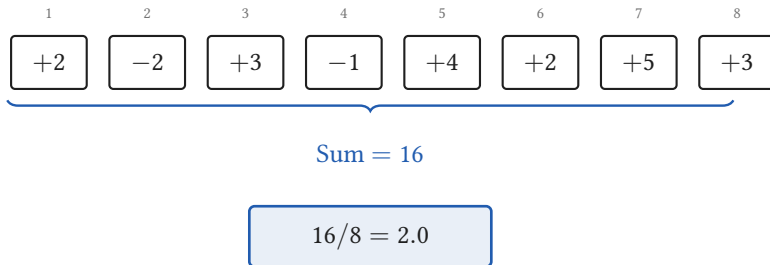


Section 2

Four workers, eight examples

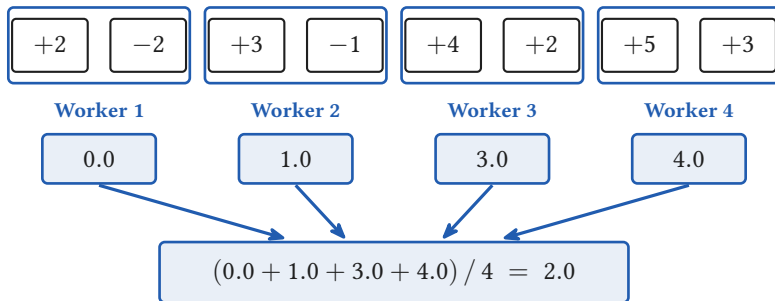
2

Eight examples, one gradient



- The single machine answer is 2.0
- Constructed for this lecture. A real gradient is a vector with one number per parameter; one number per example keeps the arithmetic hand checkable.

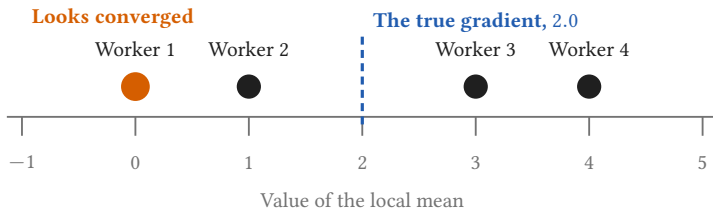
Four workers, two examples each



The same number the single machine got

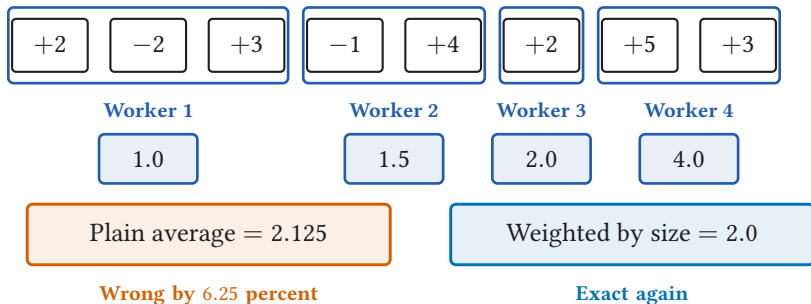
Exactly equal, not approximately: splitting the batch is a regrouping of the same sum.

Worker 1 sees zero



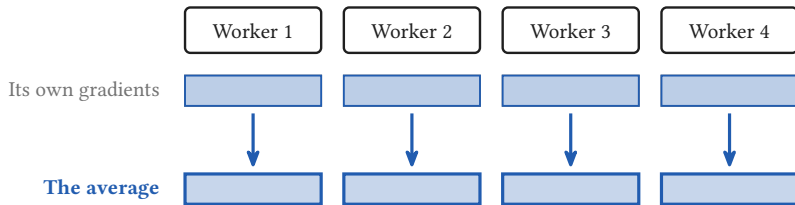
- Worker 1's local mean is zero. Its own shard looks converged.
- The global answer is 2.0, and no worker holds it
- That is why the averaging step exists, and why it cannot be skipped

Unequal shards break the identity



Every framework therefore fixes the per worker micro batch and drops or pads the ragged tail.

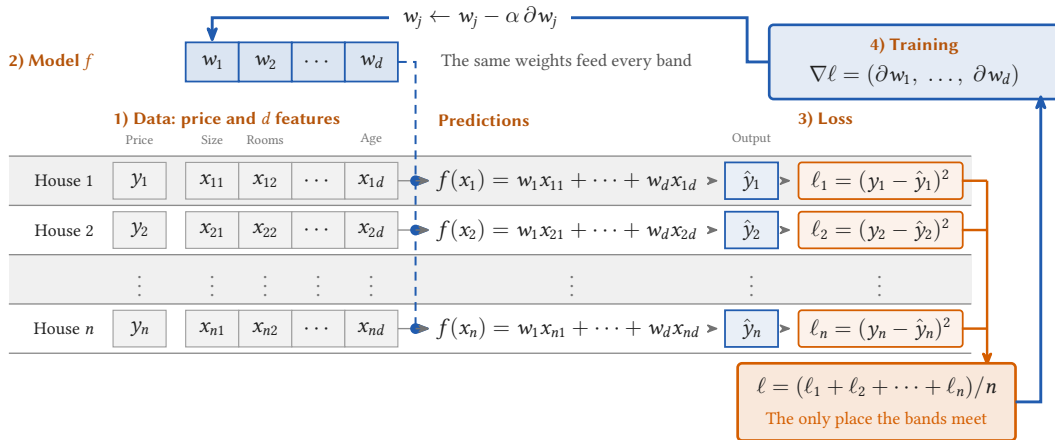
One buffer, 16.11 gigabytes



Every worker ends holding the same averaged buffer

- Apertus-8B in bf16: $2 \text{ bytes} \times 8.053 \times 10^9 \text{ parameters} = 16.11 \text{ GB}$
- Write K for the size of that buffer, and P for the number of workers
- Compute is local. This is the only part that is not.

One training step of linear regression



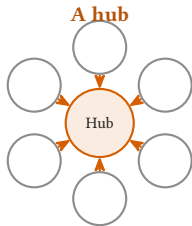


Section 3

The all-reduce

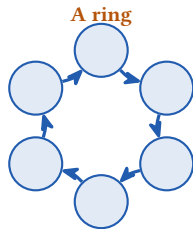
3

Two ways to average: a hub, or a ring



$P \times K$ arrives at one machine

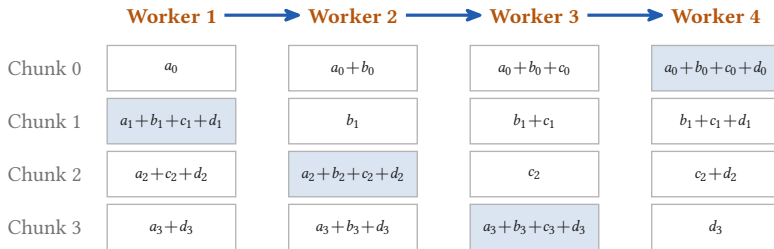
K is the gradient buffer, 16.11 GB. P is the number of workers.



$\frac{2(P-1)}{P} \times K$ leaves each worker

- The hub's traffic grows with every worker you add

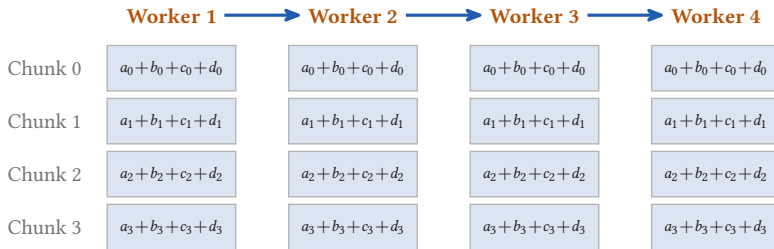
Round the ring, adding



Round 3: each worker now owns one chunk of the true sum

Chunk j is the same slice of the buffer on every worker, so a_j is worker 1's chunk j , b_j worker 2's, and worker 4 sends back to worker 1.

Round the ring again, copying



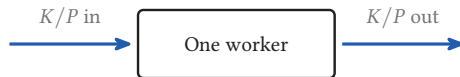
Round 6: every worker now holds the whole averaged buffer

Every worker holds the whole model, so the copies stay identical only if every worker applies the same whole average

Same ring, same traffic, one change: the arriving chunk replaces instead of adding. The four rows are four different slices, so a column of four cells is the whole averaged buffer.

Two times P minus one rounds

$$\text{bytes per worker} = \underbrace{\frac{2(P-1)}{P}}_{\text{always below 2}} \times \underbrace{K}_{\text{the whole buffer}}$$

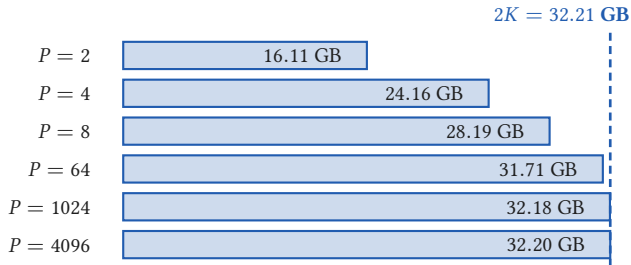


$P - 1$ rounds adding, then $P - 1$ copying

A chunk has to reach the other $P - 1$ workers, so one pass round the ring is $P - 1$ rounds, three of them at $P = 4$

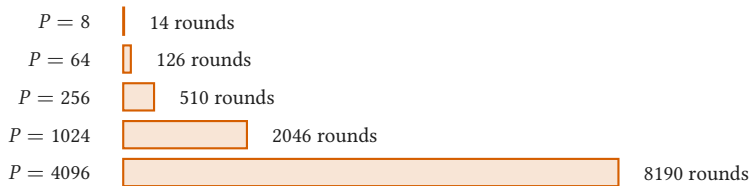
- The factor never reaches 2, so the traffic never reaches $2K$
- [Patarasuk and Yuan 2009](#) proved this is the least any all-reduce can move.

The bytes barely grow



- 512 times the workers, 14.3 percent more bytes each
- Apertus-8B gradients in bf16, so $K = 16.11$ GB.

The rounds grow linearly in P



Each round pays a latency, whatever it carries

- The bytes barely grow. The rounds grow linearly.
- Past 512 GPUs the latency no longer hides behind the backward pass
- The ring is not Baidu's invention: it is [Patarasuk and Yuan 2009](#).



Section 4

Where it stops scaling

4

One step, two terms

$T(P)$ is the wall clock time for *one training step* on P workers

$$T(P) = \underbrace{\frac{T(1)}{P}}_{\text{falls with } P} + \underbrace{\frac{2(P-1)}{P} \cdot \frac{K}{B}}_{\text{does not fall at all}}$$

Work per step: 8 million tokens $\times 6N = 3.35 \times 10^{17}$ FLOP

Per GPU rate: 990 TFLOP/s dense BF16 per GPU, at an MFU of 0.30

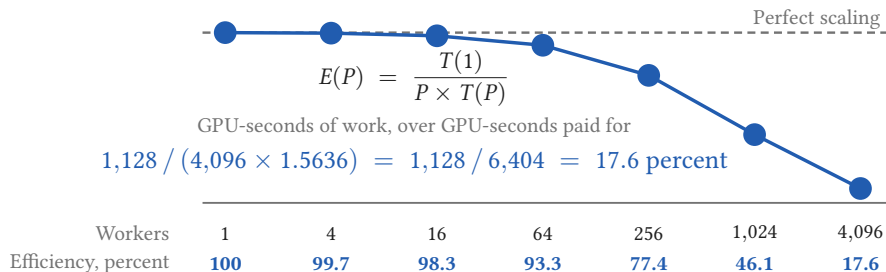
So $T(1)$ is: 1128 seconds, about 19 minutes, on one GPU

The buffer K : 16.11 GB, the gradients of Apertus-8B in bf16

The link B : 25 GB/s per GPU, Slingshot-11 on Alps

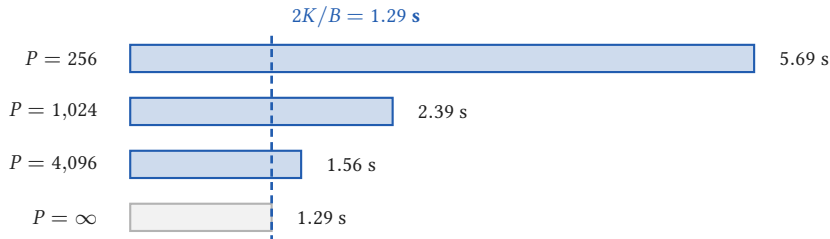
$$T(1024) = \underbrace{1128/1024}_{\text{compute}} + \underbrace{1.29}_{\text{all-reduce}} = 2.39 \text{ seconds}$$

93 percent at 64 workers, 18 at 4,096



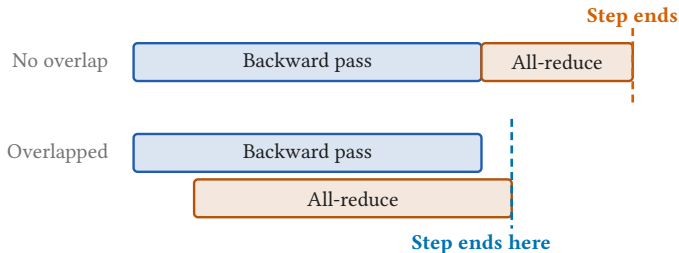
- So 82 percent of the cluster-seconds in that step were spent waiting
- At 1,024 workers, communication is 54 percent of the step
- The first term falls as one over P . The second does not fall at all.
- These are the model's numbers, charging the all-reduce after the whole backward pass.

In this model the step never falls below 1.29 seconds



- Compute over P goes to zero, and $2(P - 1)/P \times K/B$ goes to $2K/B$
- Four times the machine, 1,024 to 4,096, buys 0.83 s of a 2.39 s step
- Every worker after that, all of them together, buys 0.28 s more

18 percent modelled, 80 percent measured



- The model above charges the all-reduce after the whole backward pass
- Real systems start averaging before the backward pass has finished
- Apertus measured about 80 percent at 4,096 GPUs, not 17.6, so overlapping is worth a factor of about 4.5

Where it stops scaling

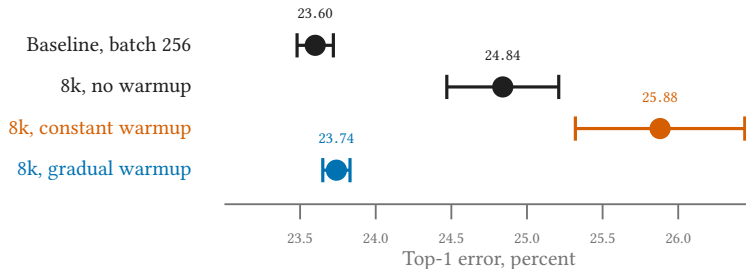
29 hours on 8 GPUs, 1 hour on 256



Ideal 32 times faster, measured 29 times: 90.6 percent

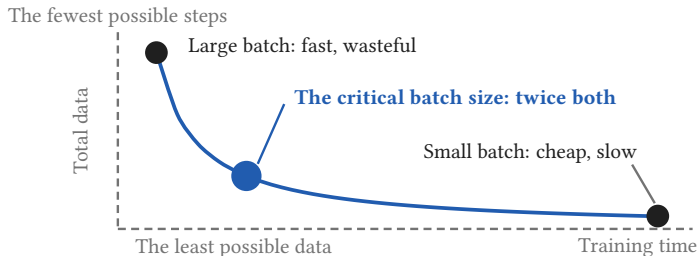
- Time per iteration rises only 12 percent while the minibatch grows 44 times
- Accuracy is stable to about 8k images and degrades rapidly beyond
- [Goyal et al. 2017](#), ResNet-50 on ImageNet.

Multiply the batch by k, the rate by k



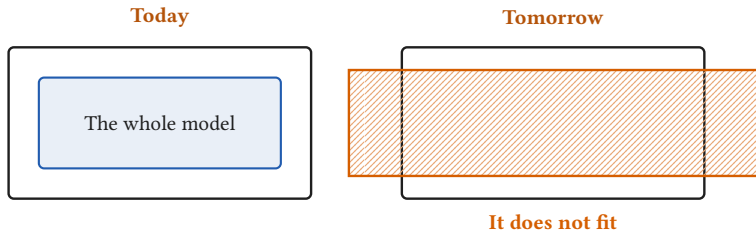
- Constant warmup is worse than no warmup at all
- Gradual warmup over five epochs recovers the baseline
- [Goyal et al. 2017](#) Table 1, five trials each. An SGD result: do not carry the linear rule to an Adam run.

Past the critical batch size



- Twice the data of the cheapest run, twice the steps of the fastest
- Measured, not derived: Billion Word 700 to 100,000 tokens
- It rises during a run, so Apertus doubled its batch mid run

Tomorrow the model does not fit



- Everything in this hour assumed one device holds the whole model
- Apertus-70B does not
- Tomorrow at 08:00: cut the model, not the batch